INSIDE ZUSTAND

State Management with Zustand

A guide to stores, selectors, middleware, and TypeScript in the pmndrs Zustand library

Covers Zustand 5.0.15 (commit `b57db4f`)

An unofficial guide, written as a worked example of the `inside-mac` documentation skills

Contents

	Preface	**4**
	About This Guide	4
	Who Should Read This Guide	4
	What to Read	4
	Supported Version	5
	Conventions	5
Chapter 1	**Introduction to Zustand**	**7**
	Read this chapter if	7
	In this chapter you will	7
	The problem	8
	Non-goals	8
	Where Zustand fits	8
	Core concepts	9
	A worked example	10
	How the pieces connect	13
	Summary	14
Chapter 2	**Stores**	**15**
	Read this chapter if	15
	In this chapter you will	15
	About stores	15
	Using stores	18
	Stores reference	21
	Summary	23
Chapter 3	**Using Stores in React**	**24**
	Read this chapter if	24
	In this chapter you will	24
	About the React integration	25
	Using stores in React	27
	React reference	30
	Summary	33
Chapter 4	**Middleware**	**35**
	Read this chapter if	35
	In this chapter you will	35

About middleware 36
Using middleware 38
Middleware reference 41
Summary 45

Chapter 5 **TypeScript** **47**
Read this chapter if 47
In this chapter you will 47
About Zustand's TypeScript design 47
Using TypeScript with Zustand 51
TypeScript reference 54
Summary 56

Chapter 6 **Testing and Patterns** **57**
Read this chapter if 57
In this chapter you will 57
Resetting stores between tests 57
Testing components that use a store 58
Slices for a large store 59
Transient updates without re-rendering 60
Summary 61

Appendix A **Glossary** **63**

Appendix B **API Summary** **65**
`zustand` 65
`zustand/react/shallow` 65
`zustand/shallow` 66
`zustand/traditional` 66
`zustand/middleware` 66
`zustand/middleware/immer` 66

Appendix C **Known Gaps and Pitfalls** **67**

Preface

About This Guide

Zustand is a small state-management library for React and other JavaScript UIs. A store holds state and the functions that change it; components read slices of that state with a selector and re-render only when the slice they read changes. This guide explains that model, the `create` and `createStore` APIs, the official middleware, and the TypeScript conventions the library uses to type them.

> **NOTE**
> This is an unofficial, independently written guide produced as an example output of the `inside-mac` skills for a coding agent. It is not affiliated with, endorsed by, or reviewed by the pmndrs organization or Zustand's maintainers. Zustand is open-source software released under the MIT License; see pmndrs/zustand for the canonical source, issue tracker, and official documentation.

Who Should Read This Guide

This guide is for developers who already know React and want to add or replace state management with Zustand. It assumes you can read TypeScript, have used React hooks, and have built at least one component that manages its own `useState`. It does not assume prior experience with Redux, Flux, or any other state library, though Chapter 1 compares Zustand with Context and Redux for readers coming from those tools.

What to Read

If you want to…	Read…	You can skip…
Decide whether Zustand fits your app	Chapter 1, Introduction to Zustand	Chapters 4–5
Learn the store model before writing code	Chapter 2, About Stores	Chapter 5

Wire a store into React components today	Chapter 3, Using Stores in React	Chapter 2's internals section
Add persistence, Redux DevTools, or Immer	Chapter 4, Middleware	Chapter 6
Type a store, middleware stack, or slice	Chapter 5, TypeScript	Chapter 4's non-TypeScript examples
Test components and stores, or split a large store	Chapter 6, Testing and Patterns	—
Look up an exact signature, option, or term	Appendix B, API Summary, or Appendix A, Glossary	Narrative chapters

Supported Version

This guide documents **Zustand 5.0.15** at commit `b57db4f` on the `pmndrs/zustand` `main` branch. Zustand 5 requires React 18 or later for the React integration (`zustand`, `zustand/react`, `zustand/traditional`) and drops the UMD build, default exports, and ES5 support that earlier versions carried. The vanilla store (`zustand/vanilla`) has no React dependency and works in any JavaScript runtime. Where a behavior changed from v4, this guide says so; it does not otherwise cover v3 or v4.

Conventions

Code samples use TypeScript and the ES module form of `import`. Samples that only illustrate the runtime API use JavaScript. Every multi-line sample is complete enough to run after you add the `zustand` import shown at its top; sample fragments that continue a previous listing say so in the surrounding text.

This guide uses three callouts:

> **NOTE**
> Optional context: background, a related pattern, or a fact that helps but is not required to proceed.

> **IMPORTANT**
> A condition you must satisfy for the surrounding procedure or type to work correctly.

> **WARNING**
> An action that can silently break re-rendering, discard data, or produce a runtime error. Warnings name the consequence and the safer alternative.

Function and hook names are set in `code font` on first use in each section and linked to their reference entry where one exists. Terms defined in the Glossary appear in **bold** the first time a chapter uses them.

CHAPTER 1

Introduction to Zustand

Zustand gives a React application one or more stores: plain JavaScript objects that hold state and the functions that change it, wrapped in a hook. Components call that hook with a selector to read a slice of the store, and Zustand re-renders only the components whose slice actually changed. There is no provider to mount and no reducer/action-type vocabulary to adopt unless you want one.

Read this chapter if

- You are evaluating Zustand for a new or existing React application.
- You have used `useState` or `useReducer` and want state that outlives one component's lifetime.
- You are coming from Redux, Context, or another store and want the vocabulary mapped over.

In this chapter you will

- see the problem Zustand solves and what it deliberately leaves out;
- learn the five core concepts — store, state, actions, selectors, and subscriptions — that the rest of this guide builds on;
- follow one complete, runnable example from store definition to rendered UI;
- see where a Zustand store sits relative to your components, in a diagram.

The problem

React's built-in state (`useState`, `useReducer`) is scoped to the component that owns it. Sharing that state with a distant component means lifting it to a common ancestor and threading it down through props, or wrapping the tree in a Context provider. Context works, but every value change re-renders every component that reads the context, regardless of which part of the value it uses, unless you split the context by hand.

Zustand stores state outside the React tree. Components subscribe directly to the store, not through a provider, and each subscription re-renders only when the specific slice it selected changes. The store survives remounts, is reachable from outside React (in event handlers, network callbacks, or plain modules), and requires no wrapping component to exist.

Non-goals

Zustand does not:

- **Replace `useState` for local UI state.** A single component's open/closed flag or input draft usually belongs in that component, not a shared store.
- **Enforce a reducer/action-type architecture.** You can colocate actions with state directly (the pattern this guide uses throughout) or opt into `redux`-style dispatch with the `redux` middleware from Chapter 4. Neither is mandatory.
- **Track fine-grained property access automatically.** Libraries such as Valtio or MobX re-render based on which properties a component actually read. Zustand re-renders based on what your selector function returns; you write the selector.
- **Provide server state, caching, or data fetching.** Pair Zustand with a data-fetching library (React Query, SWR, or your own) for request state; use Zustand for client state that isn't owned by a server response.

Where Zustand fits

	React Context	Redux (with Redux Toolkit)	Zustand
Requires a provider in the tree	Yes	Yes	No

Re-render granularity	Whole subtree, unless you split contexts by hand	Per-selector, via `useSelector`	Per-selector, via the store hook
State shape	Whatever you pass to the provider	Single normalized store, updated via reducers	Single store per `create` call; you can compose several
Update mechanism	`useState`/`useReducer` in the provider	Dispatched actions through reducers	Direct `set` calls from actions you define
Usable outside components	No — needs a component to read the context	Yes — `store.dispatch`, `store.getState()`	Yes — `store.getState()`, `store.setState()`, `store.subscribe()`
Boilerplate for one new piece of shared state	Provider + context object + hook	Slice + reducer + selectors + wiring into the root reducer	One property and one action in the store object

Read Chapter 2 for the internal reasons a Zustand store gives per-selector re-renders without a provider: it is a plain object with a subscriber list, and React's `useSyncExternalStore` connects that list to the render cycle.

Core concepts

Five terms recur through this guide. Later chapters define each precisely; this section gives the minimum you need to read the worked example below.

Store : An object created by `create` (for React) or `createStore` (framework-agnostic) that holds the current state and exposes `getState`, `setState`, and `subscribe`. A React store is also a hook: calling it inside a component subscribes that component to the store.

State : The plain data the store holds — whatever object your state creator function returns. Zustand treats state as **immutable**: you never mutate it in place; you call `set` with a new value or a partial update.

Actions : Functions that call `set` to change state. Zustand has no separate action-dispatch layer; actions are ordinary functions you define alongside your state, usually inside the same store. Chapter 6 covers the alternative of defining actions outside the store object.

Selectors : Functions of the form `(state) => someSlice`. A component passes a selector to the store hook to read only the data it needs. The store hook re-renders the component when the selector's return value changes according to `Object.is`, not when any other part of the state changes.

Subscriptions : The underlying mechanism selectors are built on. `store.subscribe(listener)` registers a function that runs after every state change; the React hook uses this internally so you rarely call it directly, except for work that must happen outside the render cycle (Chapter 6).

A worked example

This example builds a small shopping cart store, then a component tree that reads and updates it. It uses every core concept above.

```
// store.ts
import { create } from 'zustand'

type CartItem = { id: string; name: string; price: number; quantity: number }

type CartState = {
  items: CartItem[]
}

type CartActions = {
  addItem: (item: Omit<CartItem, 'quantity'>) => void
  removeItem: (id: string) => void
  clear: () => void
}

export const useCartStore = create<CartState & CartActions>()((set) => ({
  items: [],
  addItem: (item) =>
    set((state) => {
      const existing = state.items.find((i) => i.id === item.id)
      if (existing) {
        return {
          items: state.items.map((i) =>
            i.id === item.id ? { ...i, quantity: i.quantity + 1 } : i,
          ),
        }
      }
      return { items: [...state.items, { ...item, quantity: 1 }] }
    }),
  removeItem: (id) =>
    set((state) => ({ items: state.items.filter((i) => i.id !== id) })),
  clear: () => set({ items: [] }),
}))
```

The state creator function receives `set` and returns the initial state: an empty `items` array plus the three actions. `addItem` reads the current state through the updater form of `set` so it can decide whether to bump an existing line item's quantity or append a new one; `clear` uses the object form of `set` because the new state does not depend on the old state.

Now two components read different slices of the same store:

```
// CartBadge.tsx
import { useCartStore } from './store'

export function CartBadge() {
  const itemCount = useCartStore((state) =>
    state.items.reduce((sum, item) => sum + item.quantity, 0),
  )
  return <span className="badge">{itemCount}</span>
}
```

```
// CartTotal.tsx
import { useCartStore } from './store'

export function CartTotal() {
  const total = useCartStore((state) =>
    state.items.reduce((sum, item) => sum + item.price * item.quantity, 0),
  )
  return <p>Total: ${total.toFixed(2)}</p>
}
```

```
// AddToCartButton.tsx
import { useCartStore } from './store'

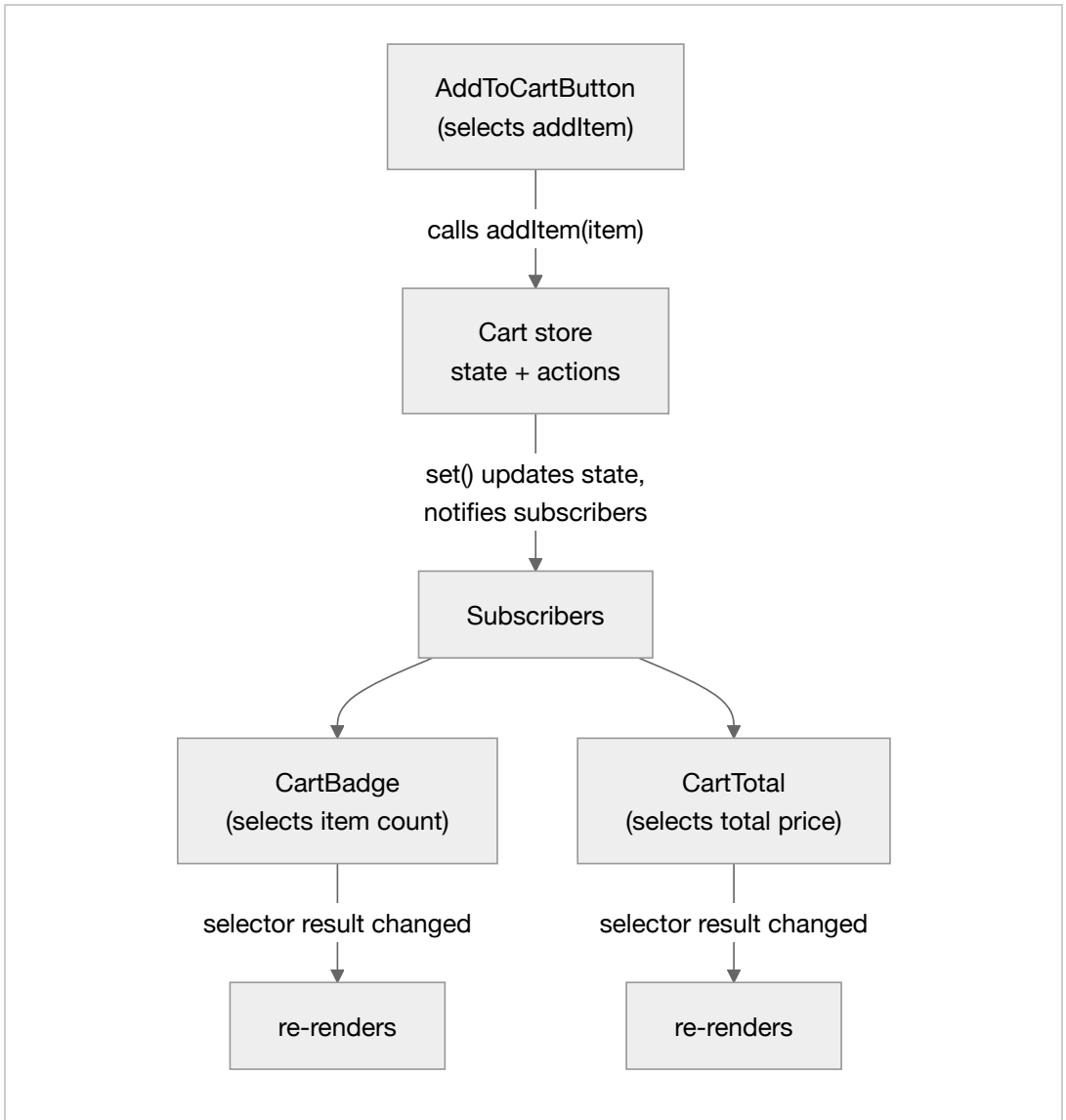
export function AddToCartButton({ id, name, price }: { id: string; name: string;
price: number }) {
  const addItem = useCartStore((state) => state.addItem)
  return <button onClick={() => addItem({ id, name, price })}>Add to cart</button>
}
```

`CartBadge` and `CartTotal` each select a derived number, not the `items` array itself. Clicking the button in `AddToCartButton` calls `addItem`, which calls `set`, which recomputes `items` and notifies every subscriber. React re-runs each subscriber's selector against the new state and re-renders only the components whose selector result changed according to `Object.is` — in this case, both `CartBadge` and `CartTotal`, because both derived numbers changed. A third component that selected, say, a customer's shipping address from a different slice of a different store would not re-render at all.

> **NOTE**
> None of these three components imports a provider or wraps the tree in one. `useCartStore` is the store and the hook; importing it is enough to read from or write to it anywhere in the component tree, or outside it.

How the pieces connect

Figure 1-1: A component subscribes to the store through a selector; only components whose selected slice changed re-render when an action runs.



No component holds a reference to another. Each one holds a reference to the store, selects the slice it needs, and reacts independently. Chapter 2 opens the store itself to show how `set`, `subscribe`, and this notification path are implemented.

Summary

- A Zustand store is a plain object with `getState`, `setState`, and `subscribe`; the React version of `create` also returns a hook.
- State is immutable data; actions are functions, usually defined alongside the state, that call `set` to produce new state.
- Selectors decide what a component reads and, by extension, when it re-renders; `Object.is` compares selector output across renders.
- Zustand does not require a provider, does not prescribe a reducer architecture, and does not fetch or cache server data — pair it with the tools that do.
- Continue to Chapter 2 to see the store's internals, or skip to Chapter 3 to focus on the React integration.

CHAPTER 2

Stores

A Zustand store is the object `createStore` returns: a closure over one piece of state, a set of listener functions, and four methods that read, write, and subscribe to that state. Every other API in this guide — the React hook from `create`, every middleware in Chapter 4 — wraps or composes this object. Understanding it precisely makes the rest of the library predictable.

Read this chapter if

- You want to know exactly what `set` does before you rely on its merge behavior.
- You are building a store that other code (middleware, a non-React UI, a test) will read or write directly through `getState` / `setState`, not only through the React hook.
- You hit a bug involving stale state, a missed re-render, or state that didn't merge the way you expected.

In this chapter you will

- see the vanilla store's four methods and what each guarantees;
- learn the merge and replace semantics of `setState`, including where they differ for object and non-object state;
- write a vanilla store, add actions, add an asynchronous action, and read the store from outside a React component;
- get the complete reference for `createStore`, `StoreApi`, and each of its four methods.

About stores

The shape of a store

`createStore` takes one function — the **state creator** — and returns a `StoreApi<T>`:

```
interface StoreApi<T> {
  setState: (partial: T | Partial<T> | ((state: T) => T | Partial<T>), replace?:
boolean) => void
  getState: () => T
  getInitialState: () => T
  subscribe: (listener: (state: T, prevState: T) => void) => () => void
}
```

Internally, the store closes over three private values: the current `state`, the `initialState` captured once at creation, and a `Set` of listener functions. `setState`, `getState`, and `subscribe` are the only way to read or change `state`; nothing outside the closure can reach it directly.

> **IMPORTANT**
> The state creator function receives `set`, `get`, and `store` as arguments, but none of the three is safe to call while the creator function itself is running — only afterward, from actions the creator returns. Calling `get()` or `store.getState()` during initialization returns `undefined`; calling `store.getInitialState()` during initialization throws, because `initialState` has not been assigned yet. Do the minimum work in the creator (return the initial object) and put logic that needs to read state in the actions you attach to it.

`setState` merge and replace semantics

Calling `setState(partial)` does three things, in order:

1. If `partial` is a function, calls it with the current state to get the next value; otherwise uses `partial` directly.
2. Compares the result to the current state with `Object.is`. If they are the same reference, `setState` does nothing — no merge, no listener notifications.
3. Otherwise, produces the new state and notifies every listener with `(newState, previousState)`.

Step 3 is where merge behavior is decided. By default, Zustand treats state as **mergeable**:

- If the next value is not an object (a number, string, array, `null`, and so on), Zustand **replaces** the state with it outright.

- If the next value is an object, Zustand **shallow-merges** it onto the current state with `Object.assign({}, state, nextState)` — one level deep, like the object spread `{ ...state, ...nextState }`.

Pass `true` as the second argument to force a full replacement instead of a merge, even for object state:

```
store.setState({ count: 0 }, true)
```

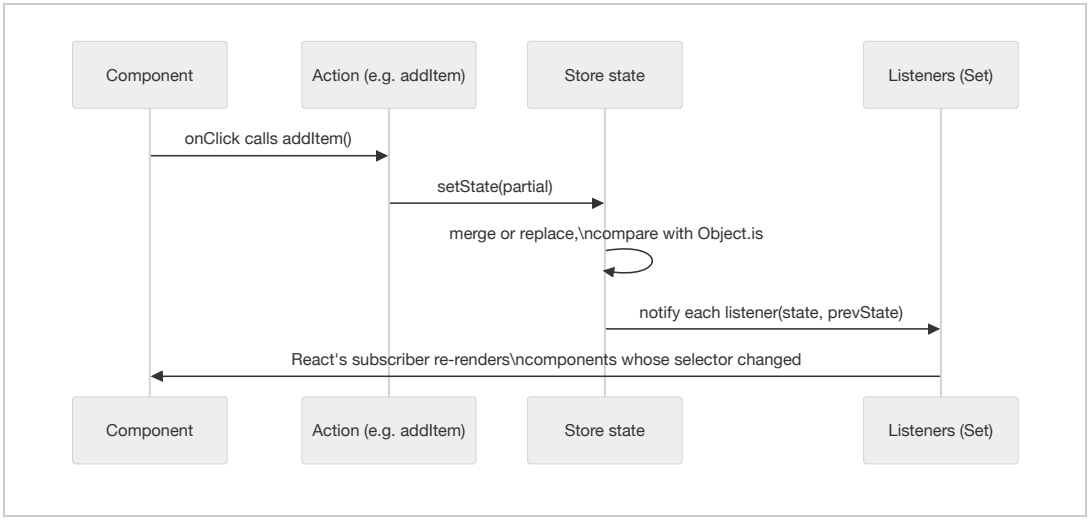
WARNING
A full replace with an incomplete object discards every key not present in the replacement. If your state also holds actions or other fields, `setState({ count: 0 }, true)` deletes them. Use `replace: true` only when you are passing a complete state object, typically the result of `getInitialState()` (see Resetting a store below) or a fully-formed object from a reducer.

NOTE
The merge is shallow. `setState({ nested: { count: 1 } })` replaces the entire `nested` object; it does not merge into whatever `nested` held before. For state more than one level deep, either spread each level by hand (`{ nested: { ...state.nested, count: 1 } }`) or use the `immer` middleware from Chapter 4, which lets you write direct-looking mutations that produce a correct immutable update.

Listeners and notification order

`subscribe(listener)` adds `listener` to the store's internal `Set` and returns an unsubscribe function that removes it. Every call to `setState` that actually changes the state (step 2 above did not short-circuit) invokes every current listener once, synchronously, in the order they were added, passing the new and previous state. There is no batching, deduplication across multiple `setState` calls, or async scheduling inside the store itself — if you call `setState` three times in a row, listeners run three times. React's `useSyncExternalStore`, which the React integration uses internally, handles batching renders on the React side.

Figure 2-1: Calling an action funnels through `set`, which updates state and synchronously notifies every subscriber, including the React hook's internal subscription.



`getInitialState`

`getInitialState()` returns the exact object the state creator produced on the store's first call, before any `setState` call ever ran. Zustand captures it once, at creation, and never updates it. Its main use is resetting a store (below); it is not a live snapshot of "the state before the most recent change" — for that, use the `prevState` argument a listener receives.

Using stores

Create a store and add actions

The recommended way to type a store is the curried form `createStore<T>()(initializer);` the extra `()` lets TypeScript infer everything the initializer returns while you still supply `T` for the state shape (Chapter 5 explains why this workaround exists). Actions live alongside state in the object the initializer returns:

```
import { createStore } from 'zustand/vanilla'

type CounterState = {
  count: number
}
```

```
type CounterActions = {
  increment: (by?: number) => void
  reset: () => void
}

export const counterStore = createStore<CounterState & CounterActions>()(
  (set, get, store) => ({
    count: 0,
    increment: (by = 1) => set((state) => ({ count: state.count + by })),
    reset: () => set(store.getInitialState()),
  }),
)
```

`reset` calls `store.getInitialState()`, which is available here because this code runs inside an action, after the store finished initializing — not inside the creator function itself.

Add an asynchronous action

An action is an ordinary function; nothing about `set` requires it to run synchronously. A `fetch`-backed action calls `set` once to record that a request started, awaits the request, then calls `set` again with the result:

```
type UserState = {
  user: { id: string; name: string } | null
  status: 'idle' | 'loading' | 'error'
}

type UserActions = {
  loadUser: (id: string) => Promise<void>
}

export const userStore = createStore<UserState & UserActions>()((set) => ({
  user: null,
  status: 'idle',
  loadUser: async (id) => {
    set({ status: 'loading' })
    try {
      const response = await fetch(`/api/users/${id}`)
      if (!response.ok) throw new Error(`Request failed: ${response.status}`)
      const user = await response.json()
      set({ user, status: 'idle' })
    } catch {
      set({ status: 'error' })
    }
  },
}))
```

Each `set` call is a separate, synchronous notification to listeners — the store does not know or care that the two calls are part of one logical operation. Components that select `status` see `'loading'` and then `'idle'` or `'error'` as two distinct renders, which is what lets a loading spinner show correctly in between.

Read and write from outside React

Because a vanilla store is just an object, any module can import it and call its methods directly, with no hook and no component:

```
import { counterStore } from './counter-store'

console.log(counterStore.getState().count) // 0

counterStore.getState().increment(5)
console.log(counterStore.getState().count) // 5

const unsubscribe = counterStore.subscribe((state, prevState) => {
  console.log(`count changed from ${prevState.count} to ${state.count}`)
})

// later, when you no longer need updates:
unsubscribe()
```

This is the basis for reading Zustand state in non-React code: route loaders, WebSocket handlers, analytics listeners, or plain utility modules. Chapter 3 covers `useStore`, the hook that connects a vanilla store like this one to React's render cycle without changing anything about the store itself.

Resetting a store to its initial state

Combine `getInitialState()` with `setState(..., true)` to reset every field, including nested objects, back to what the creator originally returned:

```
reset: () => set(store.getInitialState(), true)
```

Passing `true` matters here: `getInitialState()` returns a complete state object, including actions, so a full replace is safe and correct. Without `true`, the shallow merge would still work for a flat state shape, but explicit replacement is clearer about intent and correct even if the state later grows nested fields.

Stores reference

`createStore(stateCreatorFn)`

```
function createStore<T>(): (
  stateCreatorFn: StateCreator<T, [], []>,
) => StoreApi<T>
function createStore<T, Mos extends [StoreMutatorIdentifier, unknown][] = []>(
  stateCreatorFn: StateCreator<T, [], Mos>,
): Mutate<StoreApi<T>, Mos>
```

Creates a vanilla store. Import it from `zustand/vanilla` (also re-exported from `zustand`).

Parameters

Name	Type	Description
`stateCreatorFn`	`StateCreator<T, [], []>`	Called once, synchronously, during `createStore`. Receives `set`, `get`, and `store`; must return the initial state object.

Returns

A `StoreApi<T>` (or, when middleware is present, the mutated type the middleware adds — see Chapter 4).

Behavior

- Calls `stateCreatorFn` exactly once, synchronously, before `createStore` returns.
- Captures the returned object as both the live state and `initialState`.
- Does not call any middleware or React code; `zustand/vanilla` has no React dependency.

See also: `create`, in Chapter 3, for the React-bound version that also returns a hook.

`StoreApi<T>`

Member	Signature	Description
`getState`	`() => T`	Returns the current state synchronously. Never throws.
`getInitialState`	`() => T`	Returns the state object the creator function returned. Throws if called from inside the creator function itself, before initialization finishes.

`setState`	`(partial, replace?) => void`	Updates state. See merge and replace semantics above.
`subscribe`	`(listener) => () => void`	Registers `listener`, called synchronously after every state-changing `setState`. Returns an unsubscribe function.

`setState(partial, replace?)`

```
setState(partial: T | Partial<T> | ((state: T) => T | Partial<T>), replace?: false):
void
setState(state: T | ((state: T) => T), replace: true): void
```

Name	Direction	Type	Required	Description
`partial`	in	`T | Partial<T> | ((state: T) => T | Partial<T>)`	yes	The next state, a partial update, or a function that computes one from the current state.
`replace`	in	`boolean`	no	When `true`, replaces state instead of merging. TypeScript then requires `partial` to be a *complete* `T`, not a `Partial<T>`, to prevent accidentally deleting keys.

Behavior: Computes the next value, skips the update entirely if it is `Object.is`-equal to the current state, otherwise merges (default) or replaces (`replace: true`) and notifies every listener with `(nextState, previousState)`.

Errors: Does not throw for any input; a function `partial` that throws propagates that error to the caller of `setState` before any listener runs.

`subscribe(listener)`

Name	Direction	Type	Required	Description
`listener`	in	`(state: T, prevState: T) => void`	yes	Called after each state-changing `setState`, synchronously, in registration order.

Returns: `() => void` — call it to remove the listener. Safe to call more than once; the second call is a no-op.

Special considerations

- **Concurrency:** Listeners run synchronously on the same call stack as the `setState` call that triggered them. A listener that itself calls `setState` re-enters this same notification path.

- **Ownership:** The store owns the listener set; you do not need to track subscriptions yourself beyond keeping the returned unsubscribe function if you intend to call it later.

See also: `subscribeWithSelector`, in Chapter 4 — adds a selector and an equality function to `subscribe` so a listener fires only when a chosen slice changes.

Summary

Mental model

A store is state plus listeners in a closure. `setState` updates the state and synchronously notifies every listener; nothing about the store depends on React.

Common path

1. Call `createStore<T>()(initializer)` with actions colocated in the returned object.
2. Call actions to update state; they call `set` internally.
3. Read `getState()` or subscribe with `subscribe()` from anywhere, React or not.
4. Reset with `setState(getInitialState(), true)` when you need to restore defaults.

Invariants

- **Must not** call `get()`, `store.getState()`, or `store.getInitialState()` from inside the state creator function itself — only from actions, after initialization.
- **Must** pass a complete state object when calling `setState(value, true)`; a partial object with `replace: true` silently discards the missing keys.
- **Never** rely on `setState` batching multiple calls into one notification; each call notifies listeners independently.

Continue

- To connect a store to React components and control re-renders with selectors, read Chapter 3.
- To add persistence, Redux DevTools, or Immer-style updates on top of `setState`, read Chapter 4.

CHAPTER 3

Using Stores in React

Chapter 2 covered the vanilla store: state, `setState`, and `subscribe`. This chapter covers the layer Zustand adds on top for React — `create`, the hook it returns, and the selector-based re-render rules that make that hook efficient. It also covers the two variations most apps eventually need: a custom equality function, and a store scoped to part of the component tree instead of shared globally.

Read this chapter if

- You are building React components that read or write a Zustand store.
- A component re-renders more often than you expect, or not when you expect it to.
- You need one store instance per component subtree — a modal, a form, a tab — instead of one global store.

In this chapter you will

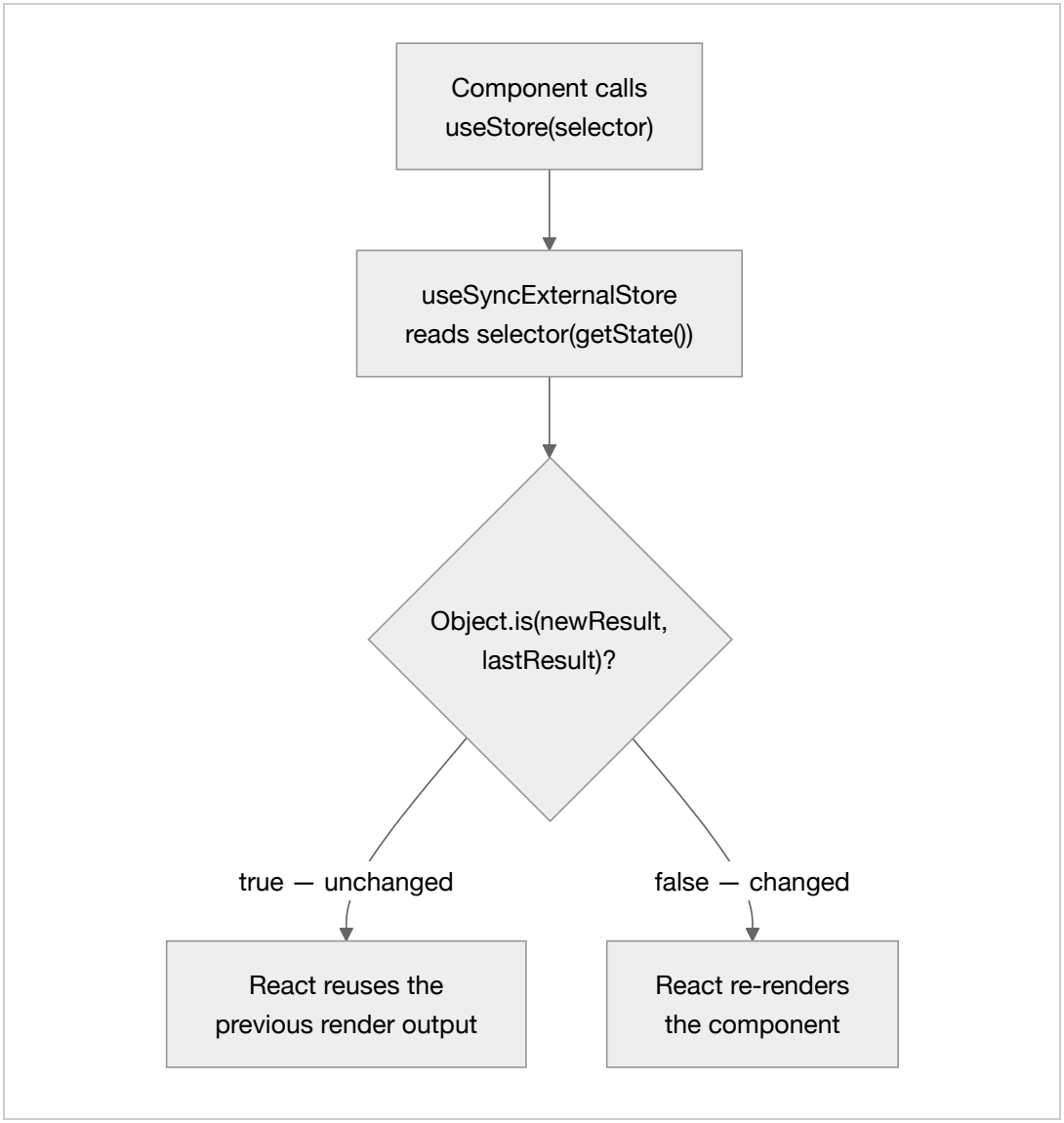
- see how `create` connects a store to React's render cycle through `useSyncExternalStore`;
- learn the exact rule that decides whether a selector triggers a re-render, and the two ways to loosen it (`useShallow`, `createWithEqualityFn`);
- scope a store to a subtree with React Context, for per-instance state or SSR-safe stores;
- get reference entries for `create`, `useStore`, `useShallow`, and the `zustand/traditional` equality-function hooks.

About the React integration

`create` does two things: it calls `createStore` internally to build a vanilla store, then wraps that store's `getState` and `subscribe` in a React hook built on `React.useSyncExternalStore`. `useSyncExternalStore` is the hook React provides specifically for subscribing to state that lives outside React — exactly what a Zustand store is. It requires three things: a way to subscribe, a way to read the current value, and (for concurrent rendering and SSR consistency) a way to read the value used for the initial server-rendered snapshot. Zustand supplies `subscribe`, `() => selector(getState())`, and `() => selector(getInitialState())` for those three roles respectively.

The object `create` returns is the hook itself, with `setState`, `getState`, `getInitialState`, and `subscribe` attached to it directly — the same `StoreApi<T>` members from Chapter 2. That is why `useCartStore.getState()` and `useCartStore((state) => state.items)` are both valid: the first calls the store API attached to the hook; the second calls the hook.

Figure 3-1: Every render of a component that calls the store hook re-evaluates its selector; only a changed result triggers React to re-render.



Using stores in React

Create and read

`create<T>()(initializer)` returns a hook. Call it with no arguments to read the entire state, or with a selector to read one slice:

```
import { create } from 'zustand'

type BearState = {
  bears: number
  increment: (by: number) => void
}

const useBearStore = create<BearState>()((set) => ({
  bears: 0,
  increment: (by) => set((state) => ({ bears: state.bears + by })),
}))

function BearCounter() {
  const bears = useBearStore((state) => state.bears)
  return <h1>{bears} bears around here…</h1>
}

function Controls() {
  const increment = useBearStore((state) => state.increment)
  return <button onClick={() => increment(1)}>Add a bear</button>
}
```

`BearCounter` re-renders only when `bears` changes; `Controls` never re-renders after mount, because the `increment` function reference is stable — it is created once, when the store is created, and never replaced.

The re-render rule, precisely

A selector's result is compared to its previous result with `Object.is`, the same comparison `useState` and `useMemo` use. Primitives compare by value; objects, arrays, and functions compare by reference. This has one direct consequence that surprises newcomers: a selector that **constructs** a new object or array on every call always produces a new reference, so it always looks "changed," even when its contents are identical.

```
// Re-renders on every store update, even unrelated ones —
// { bears, fish } is a new object reference each call.
const { bears, fish } = useJungleStore((state) => ({
  bears: state.bears,
  fish: state.fish,
}))
```

WARNING

Since v5, Zustand requires selector output to be referentially stable across calls with the same state, matching React's own expectations for external stores. A selector that returns a new object or array *and* is read with a dependency that re-triggers the read (for example, inside another piece of derived state) can throw `Maximum update depth exceeded` instead of merely over-rendering. Prefer one selector per primitive value, or wrap a multi-value selector in `useShallow`.

The direct fix is usually to call the hook once per value:

```
const bears = useJungleStore((state) => state.bears)
const fish = useJungleStore((state) => state.fish)
```

`useShallow`

When you need several values bundled into one object — for props, for a `useEffect` dependency array, or for readability — wrap the selector in `useShallow` instead of splitting every call:

```
import { useShallow } from 'zustand/react/shallow'

const { bears, fish } = useJungleStore(
  useShallow((state) => ({ bears: state.bears, fish: state.fish })),
)
```

`useShallow` memoizes the selector's result: it keeps the previous return value and reuses it whenever the new value is shallowly equal to it (same keys, `Object.is`-equal values one level deep), so `Object.is` sees the *same* reference across renders unless a selected field actually changed. `useShallow` works on arrays, `Map`, and `Set` in addition to plain objects, using the same shallow-equality function exported as `zustand/shallow`.

`createWithEqualityFn` and `useStoreWithEqualityFn`

`useShallow` covers shallow equality. If you need a different comparison — deep equality for a specific slice, or a domain-specific equality check — use `zustand/traditional`, which exposes a hook variant that takes an explicit equality function as a third argument:

```
import { createWithEqualityFn } from 'zustand/traditional'

const useBearStore = createWithEqualityFn<BearState>()(
  (set) => ({
    bears: 0,
    increment: (by) => set((state) => ({ bears: state.bears + by })),
  }),
  Object.is, // default equality function for every call, unless overridden per-call
)

// override per call:
const bears = useBearStore((state) => state.bears, (a, b) => a === b)
```

`createWithEqualityFn` requires the `use-sync-external-store` package (a peer dependency) because it is built on `useSyncExternalStoreWithSelector`, the selector-and-equality-function variant of React's hook, rather than the plain `useSyncExternalStore` that `create` uses. Reach for it when you are migrating code that used a custom equality function with v4's `create`, or when `useShallow`'s shallow comparison genuinely is not the comparison you need; for new code, prefer `useShallow` on individual selectors, since it composes with the standard `create` hook.

Scoping a store with React Context

`create` produces one shared, module-level store. Some UI needs one store instance per mounted component — a form, a modal, or a row in a list — so that two instances of the same component do not share state. Combine the vanilla `createStore` from Chapter 2 with a factory function and a React Context provider:

```
import { type ReactNode, createContext, useContext, useState } from 'react'
import { createStore, useStore } from 'zustand'

type FormState = { values: Record<string, string> }
type FormActions = { setField: (name: string, value: string) => void }
type FormStore = FormState & FormActions

function createFormStore() {
```

```
values: {},
    setField: (name, value) =>
      set((state) => ({ values: { ...state.values, [name]: value } })),
  }))
}

const FormStoreContext = createContext<ReturnType<typeof createFormStore> | null>
(null)

function FormProvider({ children }: { children: ReactNode }) {
  const [store] = useState(createFormStore)
  return <FormStoreContext.Provider value={store}>{children}
</FormStoreContext.Provider>
}

function useFormStore<T>(selector: (state: FormStore) => T): T {
  const store = useContext(FormStoreContext)
  if (!store) throw new Error('useFormStore must be used within a FormProvider')
  return useStore(store, selector)
}
```

`useState(createFormStore)` creates exactly one store per mounted `FormProvider`, on first render, and reuses it for the provider's lifetime. `useStore` — the plain hook exported from `zustand`, not `create` — connects that vanilla store instance to React the same way `create`'s internal hook does, but for a store you built and scoped yourself. Two `<FormProvider>` instances in the same tree get two independent stores; unmounting a provider drops its store for garbage collection.

> **NOTE**
> This is also the pattern to use in server-rendered applications when state must not leak between requests: create the store per request (or per `FormProvider` mount) instead of at module scope, so one user's server-rendered state can never be read by another user's request.

React reference

`create(stateCreatorFn)`

```
function create<T>(): <Mos extends [StoreMutatorIdentifier, unknown][] = []>(
  stateCreatorFn: StateCreator<T, [], Mos>,
) => UseBoundStore<Mutate<StoreApi<T>, Mos>>
function create<T, Mos extends [StoreMutatorIdentifier, unknown][] = []>(
```

```
  stateCreatorFn: StateCreator<T, [], Mos>,
): UseBoundStore<Mutate<StoreApi<T>, Mos>>
```

Creates a store and returns it as a React hook with the `StoreApi` methods attached.

Name	Direction	Type	Required	Description
`stateCreatorFn`	in	`StateCreator<T, [], []>`	yes	Same contract as `createStore`'s initializer (Chapter 2): returns the initial state, cannot read state synchronously during its own execution.

Returns: `UseBoundStore<StoreApi<T>>` — callable with no arguments (returns the whole state), or with a selector `(state: T) => U` (returns `U`, subscribing only to changes in `U`).

Behavior: Internally calls `createStore(stateCreatorFn)`, then returns a function that calls `useStore(store, selector)`, with the store's own methods copied onto that function via `Object.assign`.

See also: `createStore` (Chapter 2) for the framework-agnostic version this wraps.

`useStore(store, selectorFn?)`

```
function useStore<T>(store: StoreApi<T>): T
function useStore<T, U>(store: StoreApi<T>, selectorFn: (state: T) => U): U
```

Name	Direction	Type	Required	Description
`store`	in	`StoreApi<T>`	yes	Any vanilla store, from `createStore` or a `create`-based hook used as an API object.
`selectorFn`	in	`(state: T) => U`	no	Defaults to the identity function, returning the whole state.

Returns: `U` (or `T` with no selector), re-rendering the calling component when the selected value changes per `Object.is`.

Behavior: Wraps `React.useSyncExternalStore(store.subscribe, () => selectorFn(store.getState()), () => selectorFn(store.getInitialState()))`. This is the hook to reach for when you already have a `StoreApi` — from `createStore` directly, or from a context-scoped store as shown above — and want the same subscription behavior `create` gives you automatically.

See also: `create`, above, for the common case of one global store that doesn't need a separate `useStore` call.

`useShallow(selectorFn)`

```
function useShallow<T, U>(selectorFn: (state: T) => U): (state: T) => U
```

Name	Direction	Type	Required	Description
`selectorFn`	in	`(state: T) => U`	yes	A selector whose result may be a new object, array, `Map`, or `Set` on every call.

Returns: A wrapped selector function with the same signature, memoized so repeated calls that are shallowly equal to the previous result return the identical previous reference.

Behavior: Keeps one internal ref of the last returned value. On each call, compares the new selector output to that ref with the same shallow-equality algorithm exported as `zustand/shallow` (one level of `Object.is` comparisons across `Object.entries`, `Map` entries, or iterable elements). Returns the cached reference on a shallow match; otherwise updates the ref and returns the new value.

Errors: None specific to `useShallow`; a selector that throws propagates the error to the calling render, as with any selector.

See also: `createWithEqualityFn` / `useStoreWithEqualityFn`, below, for equality functions other than shallow.

`createWithEqualityFn(stateCreatorFn, defaultEqualityFn?)` and `useStoreWithEqualityFn(store, selectorFn, equalityFn?)`

Exported from `zustand/traditional`. Signatures mirror `create` and `useStore` with one addition:

```
function createWithEqualityFn<T>(
  stateCreatorFn: StateCreator<T, [], []>,
  defaultEqualityFn?: <U>(a: U, b: U) => boolean,
): UseBoundStoreWithEqualityFn<StoreApi<T>>

function useStoreWithEqualityFn<T, U>(
  store: StoreApi<T>,
  selectorFn: (state: T) => U,
  equalityFn?: (a: U, b: U) => boolean,
): U
```

Name	Direction	Type	Required	Description
`defaultEqualityFn` / `equalityFn`	in	`(a: U, b: U) => boolean`	no	Defaults to `Object.is`. Called with the previous and next selector results; a component re-renders only when it returns `false`.

Behavior: Builds on `useSyncExternalStoreWithSelector` from `use-sync-external-store`, which accepts an equality function directly, rather than `React.useSyncExternalStore`, which does not.

> **IMPORTANT**
> `zustand/traditional` requires the `use-sync-external-store` package as a direct dependency — Zustand lists it as a peer dependency precisely because only this entry point needs it.

See also: `zustand/shallow`'s `shallow` function, a ready-made equality function for this hook when you want shallow comparison without wrapping the selector in `useShallow`.

Summary

Mental model

`create` is `createStore` plus a `useSyncExternalStore`-based hook. Selector output is compared with `Object.is` by default; a component re-renders exactly when that comparison is `false`.

Common path

1. Call `create<T>()(initializer)` once, at module scope, for shared state.
2. Read with one selector per value; bundle multiple values with `useShallow` when you need one object.
3. For per-instance or per-request state, build a vanilla store with a factory function and scope it through React Context with `useStore`.

Public surface

- **Hooks:** `create`, `useStore`, `useShallow`, `createWithEqualityFn`, `useStoreWithEqualityFn`
- **Functions:** `shallow` (from `zustand/shallow`)

Invariants

- **Must** keep selector output referentially stable across calls with unchanged state — return primitives, existing references, or a `useShallow`-wrapped composite.
- **Should not** call `createWithEqualityFn` for new code unless a non-shallow equality function is actually required; `useShallow` covers the common case with less setup.
- **Never** assume `useSyncExternalStore` batches unrelated `setState` calls; each state-changing `setState` is a separate notification (Chapter 2).

Continue

- To add persistence, DevTools, or Immer-based updates to a store used this way, read Chapter 4.
- To see how the curried `create<T>()(...)` form works and how to type middleware stacks, read Chapter 5.

CHAPTER 4

Middleware

Every middleware in this chapter has the same shape: a function that takes a state creator and returns a new state creator. Zustand calls this new creator instead of your original one, giving the middleware a chance to wrap `set`, add properties to the store, or intercept the initial state before `create` or `createStore` ever sees it. Nothing about this mechanism is special-cased in the library — you can write your own middleware with the same shape, and Chapter 5 shows how to type one.

Read this chapter if

- You need to persist a store to `localStorage`, `AsyncStorage`, or another storage engine.
- You want to inspect and time-travel a store with the Redux DevTools browser extension.
- You are writing deeply nested state updates by hand and want Immer's draft-mutation syntax.
- You want a listener that fires only when a specific slice changes, or a reducer/action style on top of Zustand.

In this chapter you will

- see how middleware compose by wrapping the state creator function, and why order matters;
- use `persist`, `devtools`, `immer`, `subscribeWithSelector`, `combine`, and `redux`, each with a minimal working example;
- get the complete option and method reference for each middleware.

About middleware

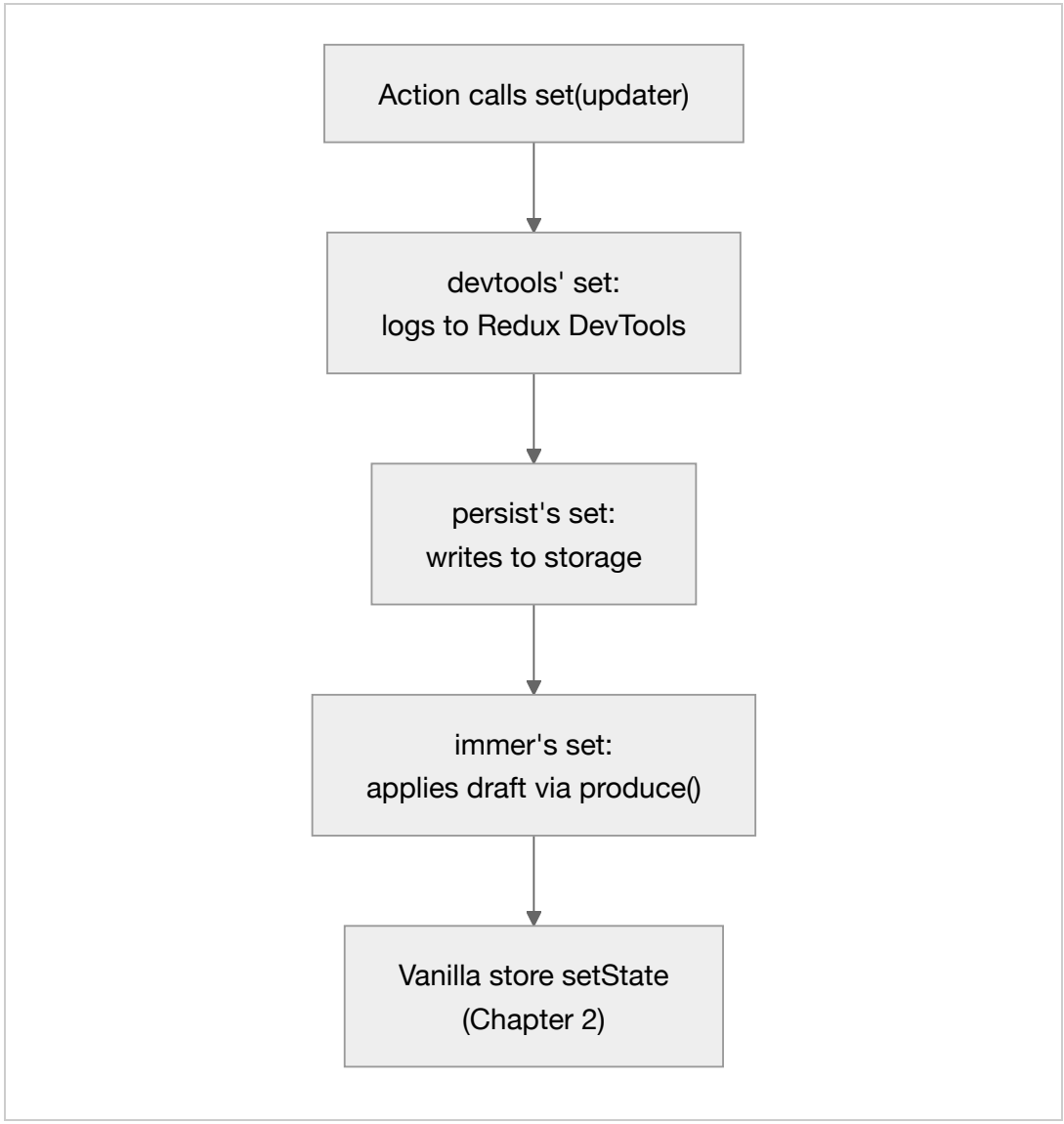
A middleware has the signature `(config: StateCreator<T, Mps, Mcs>) => StateCreator<T, Mps, [Mutator, ...Mcs]>` — it wraps a state creator and returns another one with the same three parameters, `set`, `get`, and `store`. `create` and `createStore` are unaware of middleware; they call whatever creator function you pass, and it is middleware's job to have already wrapped the real one.

Composing several middleware means nesting function calls, and nesting determines which middleware sees the *original* `set` versus a `set` another middleware has already replaced:

```
const useStore = create<State>()(
  devtools(
    persist(
      immer((set, get, store) => ({
        // your state and actions
      })),
      { name: 'my-store' },
    ),
  ),
)
```

Reading from the inside out: `immer` wraps your creator so that `set` accepts a draft-mutating function; `persist` wraps *that* result so its own `set` calls (used to write to storage) go through Immer's `set`; `devtools` wraps the outermost layer so every `set` call, including the ones `persist` makes on rehydration, gets logged to the extension.

Figure 4-1: Each middleware wraps the layer inside it; `set` calls flow outward through every wrapper between an action and the vanilla store.



> **IMPORTANT**
> Order is not cosmetic. Put `devtools` outermost so no other middleware's `setState` replacement happens after it — otherwise devtools loses track of the action name and state it wrapped. Put `immer` closest to your creator when combined with `persist`, so persisted `set` calls also accept Immer's draft-mutating form.

Using middleware

`persist`: save state to storage

`persist` writes the store to a storage engine after every `set` call and restores it — hydrates — when the store is created:

```
import { create } from 'zustand'
import { persist, createJSONStorage } from 'zustand/middleware'

type SettingsState = {
  theme: 'light' | 'dark'
  setTheme: (theme: 'light' | 'dark') => void
}

const useSettingsStore = create<SettingsState>()(
  persist(
    (set) => ({
      theme: 'light',
      setTheme: (theme) => set({ theme }),
    }),
    {
      name: 'app-settings', // storage key; must be unique
      storage: createJSONStorage(() => localStorage), // default; shown for clarity
    },
  ),
)
```

`createJSONStorage` adapts any object with `getItem` / `setItem` / `removeItem` — `localStorage`, `sessionStorage`, `AsyncStorage`, or a custom object — into the `PersistStorage` shape `persist` expects, serializing with `JSON.stringify` / `JSON.parse`.

> **NOTE**
> With a synchronous storage engine like `localStorage`, hydration completes before `create` returns, so the first render already has the persisted value. With an asynchronous engine like `AsyncStorage`, hydration finishes later, in a microtask; the first render sees the state your creator returned, not the persisted state. Use `onRehydrateStorage` or `persist.hasHydrated()` to detect the asynchronous case if your UI depends on the persisted value being present immediately.

`devtools`: inspect and time-travel with Redux DevTools

```
import { create } from 'zustand'
import { devtools } from 'zustand/middleware'

const useJungleStore = create<{ bears: number; addBear: () => void }>()(
  devtools(
    (set) => ({
      bears: 0,
      addBear: () => set((state) => ({ bears: state.bears + 1 }), undefined,
'jungle/addBear'),
    }),
    { name: 'JungleStore' },
  ),
)
```

`devtools` requires the `@redux-devtools/extension` package and the browser extension of the same name. It replaces `setState` with a version that accepts a third argument — an action name or object — and forwards every call to the extension so you can inspect a timeline of state changes and jump to any point in it. Pass `undefined` explicitly as the second (`replace`) argument when you only want to name the action, not change replacement behavior.

`immer`: draft-mutating updates

```
import { create } from 'zustand'
import { immer } from 'zustand/middleware/immer'

type TodoState = {
  todos: { id: string; text: string; done: boolean }[]
  toggle: (id: string) => void
}

const useTodoStore = create<TodoState>()(
  immer((set) => ({
```

```
toggle: (id) =>
      set((state) => {
        const todo = state.todos.find((t) => t.id === id)
        if (todo) todo.done = !todo.done
      }),
  })),
)
```

`immer` requires the `immer` package. It replaces `set` so that a function argument receives an Immer draft instead of the real state; you write direct-looking mutations on the draft, and Immer produces a correct immutable update from them. Non-function arguments to `set` work exactly as without the middleware.

`subscribeWithSelector`: subscribe to one slice

```
import { createStore } from 'zustand/vanilla'
import { subscribeWithSelector } from 'zustand/middleware'

const positionStore = createStore(
  subscribeWithSelector<{ x: number; y: number }>(() => ({ x: 0, y: 0 })),
)

const unsubscribe = positionStore.subscribe(
  (state) => state.x,
  (x, prevX) => console.log(`x moved from ${prevX} to ${x}`),
  { fireImmediately: true },
)
```

Without this middleware, `subscribe` only accepts a listener that receives the whole state on every change. With it, `subscribe` gains a second overload: a selector, a listener that receives only the selected slice, and options for a custom equality function and an immediate first call.

`combine`: infer state without the curried `create<T>()`

```
import { create } from 'zustand'
import { combine } from 'zustand/middleware'

const useBearStore = create(
  combine({ bears: 0 }, (set) => ({
    increase: (by: number) => set((state) => ({ bears: state.bears + by })),
  })),
)
```

`combine` shallow-merges an initial state object with whatever a second function returns, and Zustand infers the combined type from both arguments — so `create` does not need the curried `<T>()` form here. Reach for `combine` when you want to avoid writing an explicit state interface for a simple store; reach for the curried form when the state shape is complex enough that you want to declare and reuse it as a named type.

`redux`: reducer-and-dispatch style

```
import { createStore } from 'zustand/vanilla'
import { redux } from 'zustand/middleware'

type CounterAction = { type: 'increment' | 'decrement'; by: number }

const reducer = (state: { count: number }, action: CounterAction) => {
  switch (action.type) {
    case 'increment':
      return { count: state.count + action.by }
    case 'decrement':
      return { count: state.count - action.by }
  }
}

const counterStore = createStore(redux(reducer, { count: 0 }))

counterStore.dispatch({ type: 'increment', by: 3 })
```

`redux` wires a reducer and an initial state into a store that exposes `dispatch` both on the store API and inside the state itself, wraps every dispatched action through `set` with the action as its third argument (so `devtools`, placed outside `redux`, shows real action names), and requires no other Zustand-specific vocabulary if your team already thinks in reducers and actions.

Middleware reference

`persist(stateCreatorFn, options)`

Options (`PersistOptions<T>`)

Option	Type	Default	Description
`name`	`string`	— (required)	Storage key. Must be unique per store sharing a storage engine.

`storage`	`PersistStorage<T>`	`createJSONStorage(() => localStorage)`	Where and how state is read and written.
`partialize`	`(state: T) => PersistedState`	`(state) => state`	Selects which fields to persist.
`onRehydrateStorage`	`(state: T) => ((state?: T, error?: unknown) => void) | void`	—	Called before hydration starts; its return value is called after hydration finishes or fails.
`version`	`number`	`0`	Stored alongside the state. A mismatch with the code's `version` triggers `migrate` or discards the stored value.
`migrate`	`(persistedState: unknown, version: number) => T | Promise<T>`	returns input unchanged	Transforms an older persisted shape into the current one.
`merge`	`(persistedState: unknown, currentState: T) => T`	`{ ...currentState, ...persistedState }`	Combines the persisted value with the state the creator returned. Override for a deep merge if you have nested persisted objects.
`skipHydration`	`boolean`	`undefined`	When `true`, `persist` does not hydrate automatically; call `store.persist.rehydrate()` yourself — typically after a server-rendered app has mounted.

Store additions (`store.persist`)

Member	Signature	Description
`getOptions`	`() => Partial<PersistOptions<T>>`	Returns the current options.
`setOptions`	`(newOptions: Partial<PersistOptions<T>>) => void`	Merges new options into the current ones; can swap the storage engine at runtime.
`clearStorage`	`() => void`	Removes the stored value at `name`.
`rehydrate`	`() => Promise<void>`	Triggers hydration manually; the call `persist` makes on its own unless `skipHydration` is set.
`hasHydrated`	`() => boolean`	Non-reactive; reflects the most recent hydration's completion.

`onHydrate`	`(listener: (state: T) => void) => () => void`	Called when hydration starts. Returns an unsubscribe function.
`onFinishHydration`	`(listener: (state: T) => void) => () => void`	Called when hydration completes or fails. Returns an unsubscribe function.

`createJSONStorage(getStorage, options?)`

Name	Type	Description
`getStorage`	`() => StateStorage`	Returns an object with `getItem`, `setItem`, `removeItem`. Called once; if it throws (for example, `localStorage` is unavailable during SSR), `createJSONStorage` returns `undefined` instead of throwing.
`options.reviver`	`(key: string, value: unknown) => unknown`	Passed to `JSON.parse`.
`options.replacer`	`(key: string, value: unknown) => unknown`	Passed to `JSON.stringify`.

Mutator: `['zustand/persist', PersistedState]`

`devtools(stateCreatorFn, options?)`

Option	Type	Default	Description
`name`	`string`	—	Connection label shown in the extension.
`enabled`	`boolean`	`true` outside production, `false` in it	Toggles the extension connection.
`anonymousActionType`	`string`	inferred caller name, or `'anonymous'`	Action type used when a `set` call omits a name.
`store`	`string`	—	Groups several stores under one named connection in the extension.
`actionsDenylist`	`string | string[]`	—	Regex patterns of action types to hide from the extension's timeline; matching actions are still sent, only the display is filtered.

Store additions

Member	Signature	Description

`devtools.cleanup`	`() => void`	Disconnects from the extension. Call it when a dynamically created store (for example, a context-scoped one) is discarded.

`setState` addition: accepts a third argument, `action?: string | { type: string, ... }`, forwarded to the extension as the action that produced the new state.

Mutator: `['zustand/devtools', never]`

`immer(stateCreatorFn)`

Takes no options. Replaces `set` so a function argument receives a `Draft<T>` (from the `immer` package) instead of `T`; non-function arguments are unaffected.

Mutator: `['zustand/immer', never]`

Preconditions: Requires `immer` installed as a dependency; it is an optional peer dependency of `zustand`.

`subscribeWithSelector(stateCreatorFn)`

Takes no options. Adds an overload to `subscribe`:

```
subscribe<U>(
  selector: (state: T) => U,
  listener: (selected: U, previousSelected: U) => void,
  options?: { equalityFn?: (a: U, b: U) => boolean; fireImmediately?: boolean },
): () => void
```

Option	Type	Default	Description
`equalityFn`	`(a: U, b: U) => boolean`	`Object.is`	Comparison used to decide whether the listener fires.
`fireImmediately`	`boolean`	`false`	When `true`, calls the listener once immediately with the current selected value as both arguments.

Mutator: `['zustand/subscribeWithSelector', never]`

`combine(initialState, additionalStateCreatorFn)`

Name	Type	Description
`initialState`	T (any non-function value)	The base state.

`additionalStateCreatorFn`	`StateCreator<T, [], [], U>`	Receives `set`, `get`, `store` typed against `T`; its return value is shallow-merged onto `initialState`.

Returns: A state creator for `Omit<T, keyof U> & U`. Adds no mutator — `combine` does not change what `set`, `get`, or `store` do, only what type they carry.

> **NOTE**
> The `get` / `store` types inside `additionalStateCreatorFn` see only `T`, not the merged result, even though the real object at runtime has both. This is a deliberate, documented approximation that makes inference possible; see Chapter 5 for the precise tradeoff and when it matters.

`redux(reducerFn, initialState)`

Name	Type	Description
`reducerFn`	`(state: T, action: A) => T`	A pure reducer. Must return a complete `T`, not a partial update.
`initialState`	`T`	The starting state, before any action.

Returns: A state creator for `T & { dispatch: (action: A) => A }`.

Store additions: `dispatch` (also placed on the state itself, matching common Redux usage) and an internal `dispatchFromDevtools` flag that lets `devtools`, if present, dispatch actions received from the extension back into this store.

Mutator: `['zustand/redux', A]`

Summary

Mental model

Middleware are state-creator transformers, nested like function calls. The outermost middleware's `set` is what your actions ultimately call; each layer in between can inspect, extend, or replace it before the vanilla store ever sees the update.

Common path

1. Pick the middleware your feature needs — persistence, DevTools, Immer-style updates, granular subscriptions, inferred types, or reducer/dispatch.

2. Compose from the inside out: `immer` / `combine` closest to your creator, `persist` and `subscribeWithSelector` in the middle, `devtools` outermost.
3. Look up exact options and store additions in the reference above before shipping.

Public surface

- **Functions:** `persist`, `devtools`, `immer`, `subscribeWithSelector`, `combine`, `redux`, `createJSONStorage`
- **Store additions:** `store.persist.*`, `store.devtools.cleanup`, `store.dispatch` (with `redux`)

Invariants

- **Must** place `devtools` outermost when combining it with other `set`-wrapping middleware, or it loses track of action names and state.
- **Must** install the matching optional dependency for `persist`'s storage engine choice, `devtools`'s `@redux-devtools/extension`, and `immer`'s `immer` package — none ship as required dependencies of `zustand` itself.
- **Should not** rely on synchronous hydration timing when using an asynchronous storage engine with `persist`; check `persist.hasHydrated()` or use `onRehydrateStorage` instead.

Continue

- To type a middleware stack precisely, including one you write yourself, read Chapter 5.
- To reset a store between tests regardless of which middleware it uses, read Chapter 6.

CHAPTER 5

TypeScript

Zustand is written in TypeScript and every public function is fully typed, but two aspects of its API exist specifically to work around TypeScript inference limits: the curried `create<T>()(...)` call form, and the mutator-identifier list threaded through `StateCreator`. Neither has a runtime effect. This chapter explains what each one buys you and how to use both when writing your own store, middleware, or slice.

Read this chapter if

- You want to know why Zustand examples write `create<T>()(...)` instead of `create<T>(...)`.
- You are typing a store that uses more than one middleware.
- You are splitting a store into slices, or writing your own middleware, and need the types to follow along.

In this chapter you will

- see why `T` in `create<T>(initializer)` cannot be inferred from `initializer` alone, and how the curried form works around it;
- type a store, a middleware stack, and a slice-pattern store correctly;
- get the reference for `StateCreator`, `StoreMutators`, `Mutate`, and `ExtractState`.

About Zustand's TypeScript design

Why `create<T>()(initializer)` instead of `create<T>(initializer)`

`create`'s initializer both **produces** `T` (it returns the state) and **consumes** `T` (through the `get` parameter, which returns `T`). A type parameter that appears in both a covariant position (the return value) and a contravariant position (a function parameter) is called invariant, and TypeScript cannot infer an invariant type parameter from usage — it has no single direction to infer *from*. Left alone, `create((set, get) => ({ ... }))` infers `T` as `unknown`.

The fix is not really about `create` at all; it works around a general TypeScript limitation (microsoft/TypeScript#10571): TypeScript requires you to supply either *all* of a function's type parameters explicitly or *none* of them — you cannot supply one and let the others infer. `create<T>()` is a small function that takes no runtime arguments, returns another function, and exists solely so you can write the explicit `T` in one call and let the second call infer everything else:

```
interface BearState {
  bears: number
  increase: (by: number) => void
}

// T is explicit; the initializer's own parameter and return types are inferred.
const useBearStore = create<BearState>()((set) => ({
  bears: 0,
  increase: (by) => set((state) => ({ bears: state.bears + by })),
}))
```

> **NOTE**
> `combine` and `redux` don't need the curried form. Both middleware *produce* the state type from their own arguments (an initial state object, or a reducer's return type), so there is no invariant parameter left for `create` to infer — `create(combine(...))` and `create(redux(...))` both infer correctly without `<T>()`.

`StateCreator`'s four type parameters

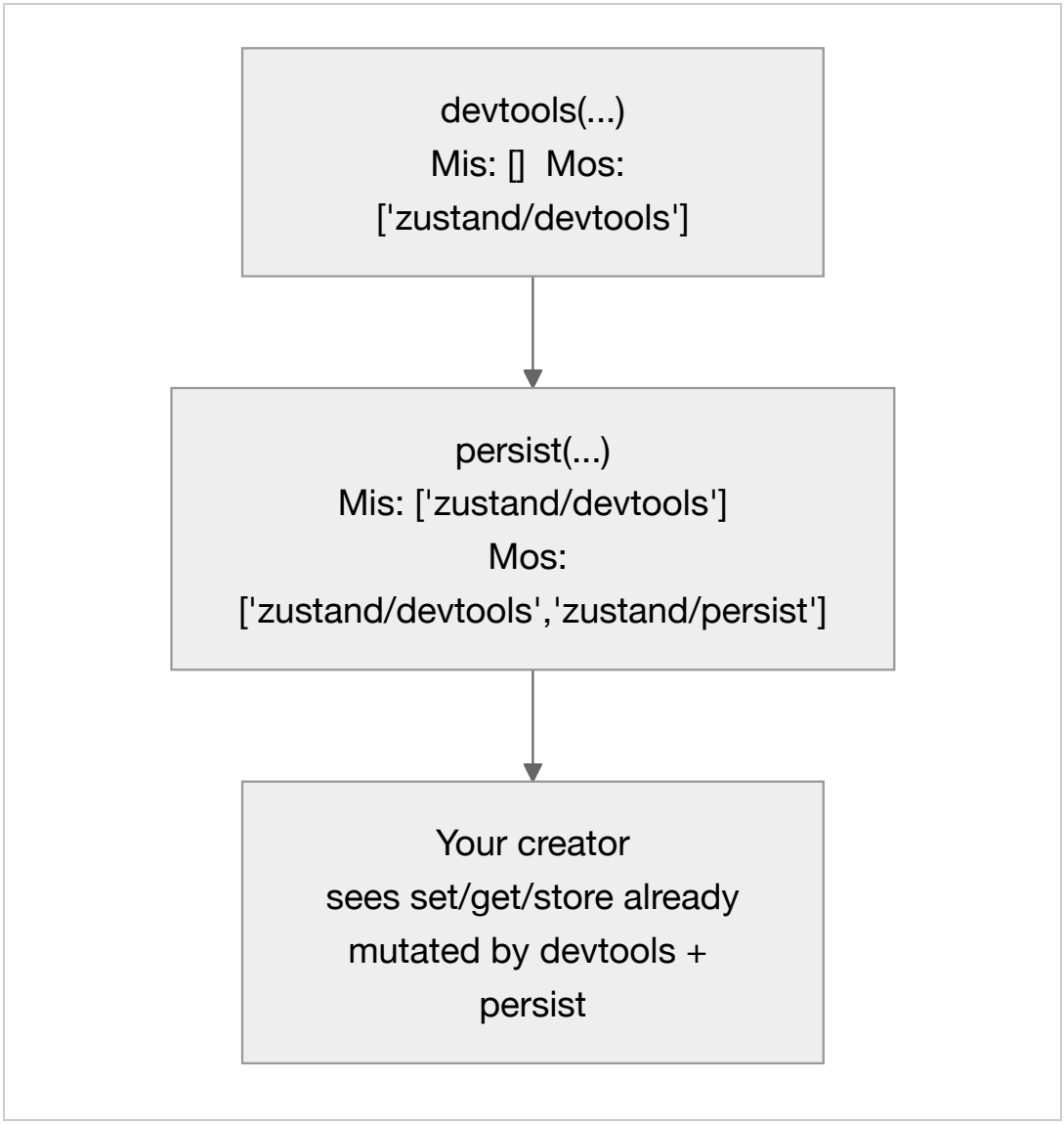
Every state creator function — the argument to `create`, `createStore`, and every middleware in Chapter 4 — has the type `StateCreator<T, Mis, Mos, U>`:

Parameter	Meaning
T	The full state type this creator ultimately participates in.

`Mis` ("mutators in")	The list of middleware mutator identifiers already applied *outside* this creator, in the order they wrap it.
`Mos` ("mutators out")	The list of mutator identifiers *this* creator (typically a middleware) adds.
`U`	What this specific creator function returns — equal to `T` for a plain store, or a slice's own shape when writing the slices pattern.

`Mis` and `Mos` are phantom type parameters: arrays of `[identifier, mutatorType]` tuples that exist only so TypeScript can compute the final, fully-mutated `set`, `get`, and `store` types — they hold no runtime value. Each middleware's mutator identifier is a fixed string, declared once in `zustand`'s ambient `StoreMutators` interface: `'zustand/persist'`, `'zustand/devtools'`, `'zustand/immer'`, `'zustand/subscribeWithSelector'`, and `'zustand/redux'`. `combine` adds no identifier, because it does not change what `set` / `get` / `store` look like — only what `T` is.

Figure 5-1: Each middleware layer consumes one mutator identifier from the list still to apply, and adds it to the list already applied, before calling the creator inside it.



Using TypeScript with Zustand

Typing a middleware stack

You do not need to restate the mutator list by hand for the built-in middleware; wrapping your creator with `devtools(persist(immer(...)))` produces the correct combined type as long as you use the curried `create<T>()` form and apply the middleware directly inside it:

```
import { create } from 'zustand'
import { devtools, persist } from 'zustand/middleware'

interface BearState {
  bears: number
  increase: (by: number) => void
}

const useBearStore = create<BearState>()(
  devtools(
    persist(
      (set) => ({
        bears: 0,
        increase: (by) => set((state) => ({ bears: state.bears + by })),
      }),
      { name: 'bearStore' },
    ),
  ),
)
```

> **IMPORTANT**
> Apply middleware directly inside `create<T>()(...)`, not through an intermediate variable such as `const withMiddleware = (f) => devtools(persist(f, options))`. TypeScript loses the contextual typing it needs through an extra layer of indirection; typing that pattern correctly requires the same higher-kinded mutator machinery middleware authors use (see below).

Writing a middleware that does not change the store type

A logging middleware that wraps `set` without adding new store members or changing `T` only needs to preserve the incoming `Mps` / `Mcs` lists:

```
import type { StateCreator, StoreMutatorIdentifier } from 'zustand'

type Logger = <
  T,
  Mps extends [StoreMutatorIdentifier, unknown][] = [],
  Mcs extends [StoreMutatorIdentifier, unknown][] = [],
>(
  f: StateCreator<T, Mps, Mcs>,
  name?: string,
) => StateCreator<T, Mps, Mcs>

type LoggerImpl = <T>(f: StateCreator<T, [], []>, name?: string) => StateCreator<T,
[], []>

const loggerImpl: LoggerImpl = (f, name) => (set, get, store) => {
  const loggedSet: typeof set = (...a) => {
    set(...(a as Parameters<typeof set>))
    console.log(...(name ? [`${name}:`] : []), get())
  }
  return f(loggedSet, get, store)
}

export const logger = loggerImpl as unknown as Logger
```

The pattern — a precisely typed public signature (`Logger`) backed by a loosely typed implementation (`LoggerImpl`), connected with one `as unknown as` cast — is the standard shape for Zustand middleware. The public type is what callers see and what makes composition typecheck; the implementation type is what you can actually write and have the compiler accept.

Writing a middleware that changes the store type

A middleware that adds a new member to the store (as `devtools` adds `cleanup` and `persist` adds `persist.*`) additionally declares its own mutator identifier and extends `StoreMutators`:

```
import type { StateCreator, StoreMutatorIdentifier, Mutate, StoreApi } from
'zustand'

type Write<T extends object, U extends object> = Omit<T, keyof U> & U

declare module 'zustand' {
  interface StoreMutators<S, A> {
    foo: Write<Cast<S, object>, { foo: A }>
  }
}
```

```
type Cast<T, U> = T extends U ? T : U

type Foo = <
  T,
  A,
  Mps extends [StoreMutatorIdentifier, unknown][] = [],
  Mcs extends [StoreMutatorIdentifier, unknown][] = [],
>(
  f: StateCreator<T, [...Mps, ['foo', A]], Mcs>,
  bar: A,
) => StateCreator<T, Mps, [['foo', A], ...Mcs]>

type FooImpl = <T, A>(f: StateCreator<T, [], []>, bar: A) => StateCreator<T, [], []>

const fooImpl: FooImpl = (f, bar) => (set, get, _store) => {
  const store = _store as Mutate<StoreApi<ReturnType<typeof f>>, [['foo', typeof
bar]]>
  store.foo = bar
  return f(set, get, _store)
}

export const foo = fooImpl as unknown as Foo
```

`declare module 'zustand' { interface StoreMutators<S, A> { ... } }` uses TypeScript's declaration merging to add one case to the interface `Mutate` consults when resolving what a chain of mutator identifiers produces on the final store type — this is the "higher-kinded mutator" mechanism the built-in middleware also use, and the reason `StoreMutators<S, A>` is declared as an empty, augmentable interface in `zustand/vanilla`.

Typing the slices pattern

Each slice creator function should declare the *full* combined state as its `T`, so it can call `get()` to read other slices, but return only its own slice as `U`:

```
import { create, type StateCreator } from 'zustand'

interface BearSlice {
  bears: number
  addBear: () => void
  eatFish: () => void
}

interface FishSlice {
  fishes: number
  addFish: () => void
}
```

```
type SharedState = BearSlice & FishSlice

const createBearSlice: StateCreator<SharedState, [], [], BearSlice> = (set) => ({
  bears: 0,
  addBear: () => set((state) => ({ bears: state.bears + 1 })),
  eatFish: () => set((state) => ({ fishes: state.fishes - 1 })),
})

const createFishSlice: StateCreator<SharedState, [], [], FishSlice> = (set) => ({
  fishes: 0,
  addFish: () => set((state) => ({ fishes: state.fishes + 1 })),
})

const useBoundStore = create<SharedState>()((...args) => ({
  ...createBearSlice(...args),
  ...createFishSlice(...args),
}))
```

If the combined store also uses middleware, add its mutator to the second type parameter of every slice's `StateCreator`, for example `StateCreator<SharedState, [['zustand/devtools', never]], [], BearSlice>`, so each slice's `set` carries the middleware-aware type.

TypeScript reference

`StateCreator<T, Mis, Mos, U>`

```
type StateCreator<
  T,
  Mis extends [StoreMutatorIdentifier, unknown][] = [],
  Mos extends [StoreMutatorIdentifier, unknown][] = [],
  U = T,
> = (
  setState: Get<Mutate<StoreApi<T>, Mis>, 'setState', never>,
  getState: Get<Mutate<StoreApi<T>, Mis>, 'getState', never>,
  store: Mutate<StoreApi<T>, Mis>,
) => U
```

The function type every state creator and every middleware conforms to. `setState` / `getState` are typed as if every mutator in `Mis` has already been applied; `store` similarly has every method and property those mutators add.

`StoreMutators<S, A>`

```
interface StoreMutators<S, A> {}
```

An intentionally empty, augmentable interface declared in `zustand/vanilla`. Each built-in middleware (and any middleware you write) adds one property to it via `declare module 'zustand'`, mapping its mutator identifier string to the type transformation it performs on `S`. `Mutate` resolves a list of mutator identifiers by looking each one up here, one at a time.

`Mutate<S, Ms>`

```
type Mutate<S, Ms> = /* recursively resolves each entry in Ms against StoreMutators
*/
```

Given a base store type `S` and a list of `[identifier, extra]` tuples `Ms`, produces the store type after applying every mutator in order. You rarely write `Mutate` directly; it appears inside `StateCreator` and inside a middleware's own implementation signature (as in the `foo` example above), not in application code.

`ExtractState<S>`

```
type ExtractState<S> = S extends { getState: () => infer T } ? T : never
```

Extracts a store's state type from the store itself — useful when you built a store with `combine` or `redux` and want the inferred state type available outside the `create` call:

```
import { create, type ExtractState } from 'zustand'
import { combine } from 'zustand/middleware'

const useBearStore = create(
  combine({ bears: 0 }, (set) => ({
    increase: (by: number) => set((state) => ({ bears: state.bears + by })),
  })),
)

type BearState = ExtractState<typeof useBearStore>
```

Summary

Mental model

`T` in `create<T>()` cannot be inferred because the initializer both produces and consumes it; the extra `()` works around a general TypeScript limitation, not a Zustand-specific one. Middleware compose their type-level effects the same way they compose at runtime — as a list of mutator identifiers resolved one at a time by `Mutate`.

Common path

1. Declare an interface for your state and actions together.
2. Write `create<YourState>()((set, get, store) => ({ ... }))`, or skip the curried form entirely when using `combine` or `redux`.
3. For slices, type each slice creator as `StateCreator<CombinedState, Mutators, [], OwnSlice>`.
4. For custom middleware, follow the public-type/implementation-type split shown above.

Public surface

- **Types:** `StateCreator`, `StoreMutators`, `StoreMutatorIdentifier`, `Mutate`, `ExtractState`

Invariants

- **Must** apply middleware directly inside `create<T>()(...)` for contextual inference to work; an intermediate wrapper function needs the full mutator-identifier machinery to type correctly.
- **Should** prefer `combine` over the curried form when a store's state is simple enough to infer from its initial value and actions alone.
- **Never** expect the curried `create<T>()` call or the mutator-identifier lists to have any runtime effect — both disappear entirely at compile time.

Continue

- To see resetting, slicing, and transient-update patterns applied together in tests, read Chapter 6.

CHAPTER 6

Testing and Patterns

This chapter is short by design: it collects the patterns you need once a Zustand store is no longer a toy — resetting state between tests, splitting a large store into slices, and reading state without triggering a React re-render.

Read this chapter if

- Your tests see state left over from a previous test.
- One store file has grown past a few hundred lines and mixes unrelated concerns.
- You need to read fast-changing state (mouse position, scroll offset, WebSocket ticks) without re-rendering a component on every change.

In this chapter you will

- reset one store, or every store in an app, between test runs;
- split a store into slices without changing how components read it;
- subscribe to a store from inside a component without causing that component to re-render.

Resetting stores between tests

Chapter 2 introduced the single-store reset: `set(store.getInitialState(), true)`. For a whole test suite, wrap `create` itself so every store it produces registers a reset function, then reset all of them in an `afterEach`:

```
// test-setup.ts
import { act } from '@testing-library/react'
import { create as actualCreate } from 'zustand'
import type { StateCreator } from 'zustand'
```

```
export const storeResetFns = new Set<() => void>()

export const create = (<T>(stateCreator: StateCreator<T>) => {
  const store = actualCreate(stateCreator)
  const initialState = store.getInitialState()
  storeResetFns.add(() => store.setState(initialState, true))
  return store
}) as typeof actualCreate

afterEach(() => {
  act(() => {
    storeResetFns.forEach((reset) => reset())
  })
})
```

Import `create` from this file instead of `zustand` in application code that a test suite exercises (a module alias or a test-runner mock, such as Jest's `__mocks__/zustand.ts` or Vitest's `vi.mock`, applies this substitution without changing import paths). Every store created with the wrapped `create` resets to its exact initial state — including nested objects and unchanged actions — after each test, so tests do not leak state into one another regardless of execution order.

> **IMPORTANT**
> Wrap `afterEach`'s reset calls in `act(...)` when any reset triggers a React state update in a currently-mounted test component; otherwise React logs an "update not wrapped in act" warning.

Testing components that use a store

Prefer asserting through the rendered UI, with React Testing Library, over reaching into a store's internal state:

```
import { act, render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { Counter } from './counter'

test('increments the visible count when clicked', async () => {
  const user = userEvent.setup()
  render(<Counter />)

  expect(await screen.findByText('1')).toBeInTheDocument()
  await user.click(screen.getByRole('button', { name: /one up/i }))
```

```
expect(await screen.findByText('2')).toBeInTheDocument()
})
```

When a test needs to assert on state a component does not render directly — a value only used to decide whether to make a network call, for example — read it with `useCounterStore.getState()` rather than adding a visible element solely to make the value assertable.

Slices for a large store

Split a store's concerns into separate functions, each with the same `(set, get, store) => ({...})` signature as a full state creator, and spread their results together in one `create` call:

```
// bear-slice.ts
import type { StateCreator } from 'zustand'

export interface BearSlice {
  bears: number
  addBear: () => void
}

export const createBearSlice: StateCreator<BearSlice & FishSlice, [], [], BearSlice>
= (set) => ({
  bears: 0,
  addBear: () => set((state) => ({ bears: state.bears + 1 })),
})
```

```
// fish-slice.ts
import type { StateCreator } from 'zustand'
import type { BearSlice } from './bear-slice'

export interface FishSlice {
  fishes: number
  addFish: () => void
}

export const createFishSlice: StateCreator<BearSlice & FishSlice, [], [], FishSlice>
= (set) => ({
  fishes: 0,
  addFish: () => set((state) => ({ fishes: state.fishes + 1 })),
})
```

```
// store.ts
import { create } from 'zustand'
```

```
import { createBearSlice, type BearSlice } from './bear-slice'
import { createFishSlice, type FishSlice } from './fish-slice'

export const useBoundStore = create<BearSlice & FishSlice>()((...args) => ({
  ...createBearSlice(...args),
  ...createFishSlice(...args),
}))
```

Components import `useBoundStore` and select from it exactly as they would from a single-file store — slicing is a source-organization pattern, not a runtime concept; there is one store, one `set`, and one set of subscribers, regardless of how many files contributed to its shape.

WARNING
Apply middleware only around the combined store in `store.ts`, not inside individual slice files. Wrapping one slice in `persist` or `devtools` gives that slice a `set` the other slices do not share, which silently breaks the assumption that every slice's actions operate on the same store.

A slice that needs to call another slice's action reads it through `get()`, the same way any action reads store state:

```
export const createBearFishSlice: StateCreator<
  BearSlice & FishSlice,
  [],
  [],
  { addBearAndFish: () => void }
> = (set, get) => ({
  addBearAndFish: () => {
    get().addBear()
    get().addFish()
  },
})
```

Transient updates without re-rendering

Some state changes too often to justify a React re-render on every update — a live cursor position, a scroll offset, an incoming metric. Read the store directly in an event handler or effect, and write to the DOM yourself, instead of selecting the value with the store hook:

```
function CursorTrail() {
  const dotRef = useRef<HTMLDivElement>(null)
```

```
useEffect(() => {
    return usePositionStore.subscribe((state) => {
      const dot = dotRef.current
      if (dot) dot.style.transform = `translate(${state.x}px, ${state.y}px)`
    })
  }, [])

  return <div ref={dotRef} className="cursor-dot" />
}
```

`subscribe` (Chapter 2) runs the listener on every state-changing `setState`, entirely outside React's render cycle — there is no selector comparison, no re-render, and no component output to reconcile. This trades React's declarative rendering for direct DOM writes; use it only for state that genuinely updates too fast for per-frame re-renders to be worthwhile, and prefer the ordinary selector-based hook for everything else.

Summary

Mental model

Testing and scaling a Zustand store both come back to the same primitives from Chapter 2: `getInitialState`, `setState`, and `subscribe`. Resetting is `setState` with the initial state; slicing is source organization around one `set`; transient updates are `subscribe` used directly, bypassing the React hook.

Common path

1. Wrap `create` in test setup to collect reset functions; reset all stores in `afterEach`.
2. Assert through rendered UI first; fall back to `getState()` only for values the UI doesn't show.
3. Split a growing store into slice files that share one `StateCreator` type parameter and one `create` call.
4. Use `subscribe` directly, writing to the DOM by hand, for state that changes faster than a re-render is worth.

Invariants

- **Must** apply middleware around the combined store, never inside an individual slice.
- **Should** reset stores between tests rather than relying on test execution order to avoid state leakage.

- **Never** use `subscribe`-driven direct DOM writes as a substitute for ordinary selectors when a normal re-render rate is acceptable — it forfeits React's reconciliation for no benefit there.

Continue

- Appendix A defines every term this guide used as jargon; Appendix B indexes every function, hook, and type by name; Appendix C lists the pitfalls this guide called out, in one place.

APPENDIX A

Glossary

action : A function that calls `set` to change a store's state. Zustand has no separate dispatch layer; actions are ordinary functions, usually defined alongside the state they change. Compare **dispatch**, the `redux` middleware's action-object variant.

dispatch : A function, added by the `redux` middleware, that takes an action object and runs it through a reducer before calling `set`. See Chapter 4.

equality function : A function `(a, b) => boolean` used in place of the default `Object.is` comparison to decide whether a selector's result has changed. See **selector**, `useShallow`, and `createWithEqualityFn` in Chapter 3.

hydration : The process by which the `persist` middleware reads previously stored state and merges it into a store, either synchronously (for storage like `localStorage`) or asynchronously (for storage like `AsyncStorage`). See Chapter 4.

listener : A function registered with `subscribe` that runs after every state-changing `setState` call. React's store hook registers a listener internally; you can also register your own. See Chapter 2.

merge (of state) : The default behavior of `setState`: a one-level-deep `Object.assign` of the next value onto the current state. Compare **replace**.

middleware : A function that wraps a state creator and returns another state creator, optionally changing what `set`, `get`, or `store` expose. See Chapter 4.

mutator identifier : A string, such as `'zustand/persist'`, that names a middleware's type-level effect in the `StoreMutators` interface, letting `StateCreator` compute the correct types for a composed middleware stack. See Chapter 5. Compare **middleware**, which is the runtime concept this identifier describes at the type level.

replace : The behavior requested by passing `true` as `setState`'s second argument: the next value becomes the entire state, discarding any keys the previous state had that the next value does not. Compare **merge**.

selector : A function `(state) => slice` passed to a store hook to read part of the state. The store hook re-renders the calling component only when the selector's result changes, by default compared with `Object.is`. See Chapter 1 and Chapter 3.

slice : A portion of a larger store's state and actions, typically defined in its own function and combined with other slices inside one `create` call. Not a separate store or a separate `set`. See Chapter 5 and Chapter 6.

state creator : The function passed to `create`, `createStore`, or any middleware; receives `set`, `get`, and `store`, and returns the initial state. See Chapter 2.

store : An object exposing `getState`, `setState`, `getInitialState`, and `subscribe` over one piece of state. A store created with `create` is also a React hook. See Chapter 2.

transient update : A state change read through `subscribe` directly, rather than through a store hook selector, so that reading it does not cause a React re-render. See Chapter 6.

APPENDIX B

API Summary

Grouped by where each name is defined. See the linked chapter for the full contract.

`zustand`

Name	Kind	Chapter
`create`	function	3
`createStore`	function	2
`useStore`	hook	3
`StateCreator`	type	5
`StoreApi`	type	2
`StoreMutators`	type	5
`StoreMutatorIdentifier`	type	5
`Mutate`	type	5
`ExtractState`	type	5

`zustand/react/shallow`

Name	Kind	Chapter
`useShallow`	hook	3

`zustand/shallow`

Name	Kind	Chapter
`shallow`	function	3

`zustand/traditional`

Name	Kind	Chapter
`createWithEqualityFn`	function	3
`useStoreWithEqualityFn`	hook	3

`zustand/middleware`

Name	Kind	Chapter
`persist`	middleware	4
`createJSONStorage`	function	4
`devtools`	middleware	4
`subscribeWithSelector`	middleware	4
`combine`	middleware	4
`redux`	middleware	4

`zustand/middleware/immer`

Name	Kind	Chapter
`immer`	middleware	4

APPENDIX C

Known Gaps and Pitfalls

Pitfall	Consequence	Fix
Mutating state directly instead of calling `set`	No listener runs; components never re-render, even though the underlying object changed.	Always produce a new value (or draft-mutate through `immer`) and pass it to `set`.
A selector that builds a new object or array on every call	The component re-renders on every store update, or — since v5 — can throw `Maximum update depth exceeded` if the new reference feeds back into another update.	Split into one selector per value, or wrap a multi-value selector in `useShallow`.
`setState(partial, true)` with an incomplete object	Silently deletes every key the replacement object omits, including actions.	Only pass `true` with a complete state object, typically from `getInitialState()`.
Reading `get()` or `store.getInitialState()` inside the state creator function itself	`get()` returns `undefined`; `getInitialState()` throws, because initialization has not finished.	Do only the minimum work in the creator; read state from the actions it returns instead.
Applying middleware inside an individual slice instead of around the combined store	That slice's `set` diverges from the rest of the store's `set`, breaking assumptions other slices and components make.	Apply middleware once, around the `create` call that combines every slice.
Assuming synchronous hydration timing with `persist` and an asynchronous storage engine	The first render reflects the creator's default state, not the persisted value, because hydration finishes later in a microtask.	Check `persist.hasHydrated()` or use `onRehydrateStorage` before relying on persisted values at first render.
Nesting `devtools` inside another `set`-wrapping middleware	Action names and payloads shown in the extension no longer match what actually ran.	Apply `devtools` as the outermost middleware in the composition.

> **NOTE**
> This guide covers Zustand 5.0.15. The experimental `unstable_ssrSafe` middleware (`zustand/middleware/ssrSafe`), which throws on server-side `setState` calls, is deliberately out of scope: its own source marks it experimental and subject to change before the project finalizes its design.